Coordina: Code Explanation

A guide to how the project works, file by file, so you can explain the code to judges, teammates, or reviewers.

---

1. What the project does (one paragraph)

Coordina is an **AI project coordinator for teams**. It checks in with each teammate (on Telegram, the mobile app, or the dashboard), reads their free-text reply ("API is connected, but I'm waiting for test credentials from Meera"), works out the real task status, and then acts on it. It marks the task **Blocked**, messages the teammate who can unblock it, refuses false "done" claims, and pushes every change live to all screens. The same logic runs for a college group project and for a company sprint.

The core loop:

```
Follow-up → Teammate reply → AI interprets it → Guardrails check it → DB update →
Relay / unblock / handoff → Live push to every screen
```

---

2. Tech stack

Backend (Python)

Technology	Used for	Where
Python 3.11+	Main language	`backend/`
FastAPI	REST API, WebSocket endpoint, dependency-injected auth	`main.py`
Uvicorn (`uvicorn[standard]`)	ASGI server, with WebSocket support	started by `start_all.py`
SQLModel (SQLAlchemy + Pydantic)	Database models and request validation	`models.py`, `database.py`
SQLite	Database file `coordinator.db`	`database.py`
OpenAI Python SDK	Client for any OpenAI-compatible LLM (Groq, Gemini)	`llm_engine.py`
Ollama (optional)	Local LLM on the laptop for offline AI	`llm_engine.py`
httpx	HTTP calls to GitHub, Ollama, Slack	`github_repo.py`, `proof.py`, `notifier.py`
python-telegram-bot v22	Telegram bot (commands, buttons, free-text replies)	`telegram_service.py`

Technology	Used for	Where
python-dotenv	Loads settings from `backend/.env`	`database.py`
HMAC-SHA256 tokens (standard library)	Signed login tokens, salted password hashing	`auth.py`
asyncio	Auto-Pilot background loop, WebSocket broadcasting	`main.py`

Frontend (JavaScript)

Technology	Used for
React 18	UI components and state
Vite 5	Dev server, build, and proxy for `/api` and `/ws` to the backend
Tailwind CSS 3 (+ PostCSS, Autoprefixer)	Styling, light and dark themes
lucide-react	Icons
qrcode.react	QR codes so phones can open the app or link Telegram
PWA manifest	Lets Android phones "Add to Home screen" as an app
WebSocket API (browser)	Live updates from the backend
Web Speech API (browser)	Voice dictation on the mobile view

External services (all optional)

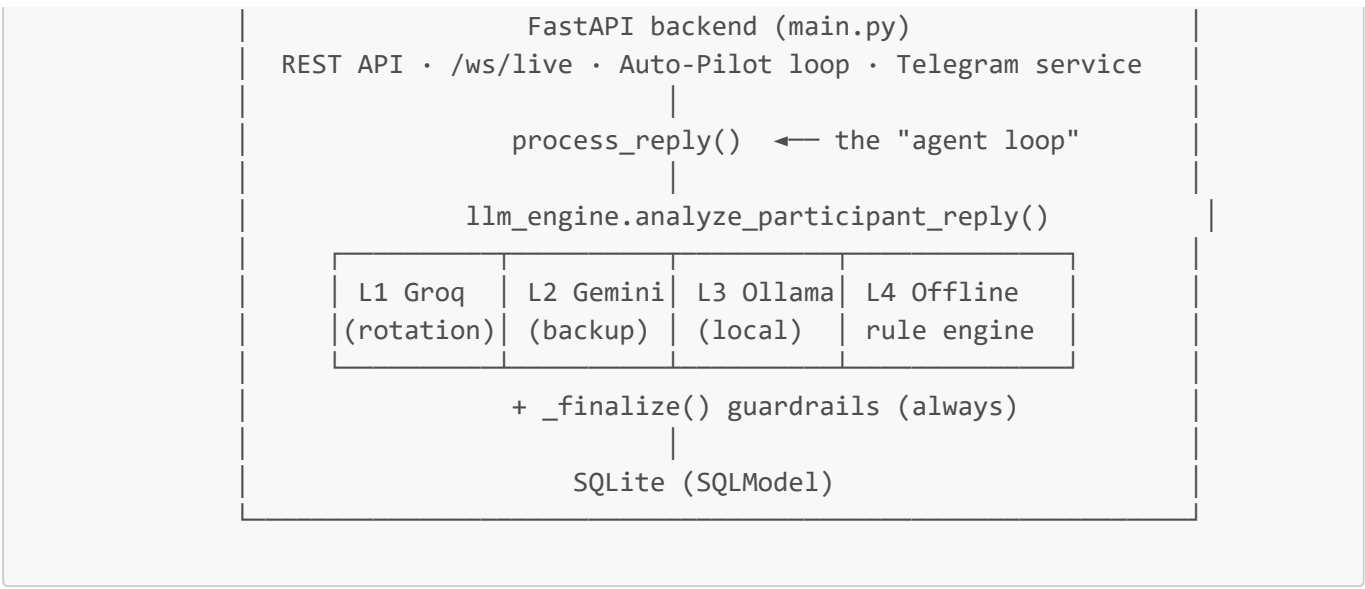
Service	Role
Groq (primary LLM)	Fast free-tier models, rotated on rate limits
Google Gemini (backup LLM)	Second cloud provider
Telegram Bot API	Check-ins and replies on teammates' phones
GitHub REST API + webhooks	Repo analysis, commit-based progress, proof of work
Slack incoming webhook	Mirrors relays and escalations to a channel

> Everything works **without any API keys**: the offline rule engine handles the AI, and Telegram runs in Simulator Mode.

---

3. Architecture





Every request that changes data triggers a `STATE_UPDATED` WebSocket event. Every open dashboard and phone then re-fetches `/api/state`, so all screens stay in sync.

---

4. Folder structure

```
ai-group-project-coordinator/
├── start_all.py          One-command launcher (backend + frontend, prints LAN
URLs for phones)
├── verify_demo.py        Runs the judged demo story against the live API and
prints pass/fail
├── README.md             Product overview, setup, demo script
├── backend/
│   ├── main.py           FastAPI app: routes, WebSocket, agent loop, Auto-Pilot,
auth, integrations
│   ├── llm_engine.py     4-layer AI engine, offline rule engine, guardrails,
reports, planners
│   ├── models.py         Database tables: User, Project, Participant, Task,
AgentLog
│   ├── database.py       SQLite engine, lightweight migrations, data backfill
│   ├── auth.py           Signed tokens, password hashing, invite codes, lead-only
guard
│   ├── telegram_service.py  Telegram bot: /start /auth /join /status /slot +
free-text replies
│   ├── notifier.py       Outbound Telegram and Slack messages (with retries)
│   ├── github_repo.py    Reads a GitHub repo: tech stack, dependencies, README,
commits
│   ├── proof.py          Proof of work: verifies commit / PR / deliverable links
│   ├── deadlines.py      Parses "Today 8 PM" style deadlines, spreads deadlines
across a timeline
│   ├── seed.py           Demo data: "College Capstone" and "Company Sprint"
presets
│   ├── self_test.py      Backend test suite (runs on a throwaway DB)
│   ├── requirements.txt
│   └── .env.example      All configuration options, documented
└── frontend/
```

```
    ├── index.html, vite.config.js, tailwind.config.js
    ├── public/manifest.json + icons   PWA install files
    └── src/
        ├── main.jsx              Entry: ErrorBoundary → Toast → Auth → App
        ├── App.jsx               Chooses login, desktop, or mobile app
        ├── hooks/useLiveState.js REST snapshot + WebSocket live updates
        ├── lib/                  api client, auth context, wording per workspace,
helpers
        └── components/           Dashboard, mobile app, report, login, modals
```

---

5. Backend, file by file

5.1 `models.py`: the data model

Five tables, defined with SQLModel:

Table	Key fields	Meaning
`User`	name, email, role (`lead`/`member`), password_hash, org_type	Someone who signs in
`Project`	name, workspace_type (`college`/`company`), invite_code, github_repo_url, strict_dependencies, start_at, due_at	One team's workspace
`Participant`	name, role, telegram_chat_id, availability, auth_pin, is_verified	A teammate inside a project
`Task`	title, owner_id, **status**, progress_note, **blocker_reason**, blocked_by_participant_id, **depends_on_task_id**, deadline, github_verified, deliverable_url, lead_approved	One unit of work
`AgentLog`	log_type, summary, details_json	Every message, AI decision, and alert (feeds the activity feed)

- **Task statuses:** `Pending`, `In Progress`, `Blocked`, `Complete`.
- **Dependencies:** `depends_on_task_id` links tasks into a chain. The guardrails use this chain to catch false "done" claims.
- **Automatic fields:** SQLAlchemy `before_insert` listeners give every new participant a unique 6-digit PIN (used to verify Telegram) and classify every new task as `github_code`, `design_docs`, or `general` with keyword regexes.

5.2 `database.py`: storage and migrations

- Creates the SQLite engine. A relative path is resolved against `backend/`.
- `_migrate()` adds new columns to old databases with `ALTER TABLE`, because SQLite's `create_all` never changes existing tables. Old demo data keeps working.
- `_backfill()` fills values for old rows: PINs, task types, and free-text deadlines converted to ISO dates.
- `get_session()` is the FastAPI dependency that gives each request a DB session.

5.3 `main.py`: the FastAPI app (the heart of the backend)

Startup (`lifespan`): creates tables, seeds demo data if the DB is empty, starts the Telegram bot, warms up the local LLM, and optionally starts Auto-Pilot.

Real-time updates

- `ConnectionManager` keeps the set of open WebSockets.
- An HTTP middleware (`broadcast_on_mutation`) sends `STATE_UPDATED` after any successful POST/PATCH, so no route needs to remember to broadcast.
- `/ws/live` also answers `ping` with `PONG` so the UI can show latency.

The agent loop: `process_reply()`

This one function runs whenever a teammate sends an update from any channel:

1. Load the project's participants and tasks.
2. Replace links in the message with `(link)` so a URL is not read as a status word.
3. Call `llm_engine.analyze_participant_reply()` to get the new status, progress note, blocker, relay target, and so on.
4. Log the raw reply and the AI's reasoning (`REPLY`, `AI_REASONING`).
5. Update the task: status, progress note, blocker reason, and who it is blocked by. Attach proof (commit or link) if there is any.
6. Update the teammate's availability if the message mentioned it ("free after 6").
7. **Contradiction:** if the AI refused a completion, log a `BLOCKER_ALERT`.
8. **Relay:** if blocked by a named teammate, message that teammate with exactly what is needed.
9. **Auto-unblock:** if the message resolves someone else's blocker ("sent the keys to Kabir"), move their task back to `In Progress` and tell them.
10. **Handoff:** if the task is now `Complete`, notify everyone whose task depends on it.
11. Reply to the sender on Telegram, commit, and return a summary of all actions.

Auto-Pilot (`run_autopilot_cycle`): a background asyncio loop. On each cycle:

- **Pending** tasks get a check-in, but only if the upstream task is done (no pointless nagging).
- **In Progress** tasks that are overdue or have had no update for a while get a follow-up.
- **Blocked** tasks past the SLA are escalated: level 1 goes to the teammate who can fix it, level 2 goes to the manager or lead role.
- Cooldowns stop the same person being pinged repeatedly. The cadence can be live demo (45 s), hourly, every 6 hours, or daily at 10 AM.

Main route groups

Group	Example endpoints
State	`GET /api/state`, `GET /api/report`
Projects and tasks	`POST /api/projects`, `POST /api/tasks`, `PATCH /api/tasks/{id}`, `/override`, `/review`, `/deadline`, `/deliverable`
Teammate updates	`POST /api/simulate-reply`, `POST /api/mobile/quick-update`, `GET /api/mobile/my-dashboard/{id}`

Group	Example endpoints
AI planning	`POST /api/ai/generate-project` (plan from a description), `POST /api/projects/from-github` (plan from a repo)
Auto-Pilot	`POST /api/autopilot/toggle`, `GET /api/autopilot/status`, `POST /api/autopilot/run-now`
Check-ins	`POST /api/ping-task/{id}`, `POST /api/ping-all-pending`, `POST /api/coordinate-slots`
Auth	`/api/auth/register-lead`, `/login-lead`, `/join-member`, `/demo`, `/me`, `/api/invites/{code}`
Integrations	`POST /api/webhooks/github`, `POST /api/import-csv`, `/api/integrations`, `/sync-github`
Demo	`POST /api/presets/load`, `POST /api/reset-demo`, `GET /api/network-info`

Interactive docs for all of these: `http://localhost:8000/docs`.

5.4 `llm_engine.py`: the AI brain

The 4-layer AI shield (`call_llm_json`): the AI never fails as a whole. It tries each layer in order:

Layer	What	Fallback behaviour
1. Primary cloud LLM	Groq, with a list of models	Rotates to the next model on a rate limit or timeout. A network error skips straight to layer 2.
2. Backup cloud LLM	Gemini (any OpenAI-compatible API)	Used when layer 1 fails
3. Local LLM	Ollama on the laptop	Works with no internet. Skipped for long jobs like whole project plans.
4. Offline Smart Engine	Regex and rule-based interpreter	Always works, instantly

Supporting pieces:

- A simple **circuit breaker** (`_cool` / `_cooling`) skips a provider for a while after it fails, so the demo doesn't wait on timeouts.
- **`repair_json()`** cleans model output: it removes `<think>` blocks and code fences and extracts the JSON.
- `LLM_FORCE_OFFLINE=1` forces layer 4 for rehearsals.

Offline Smart Engine (`_offline_analyze`): reads a reply using regexes for:

- done / negated-done ("not done yet"), blocker, progress, and resolution ("sent", "shared") words
- sarcasm ("yeah totally done lol")
- Hinglish phrases ("bas test credentials baaki hai"), translated with a small dictionary
- contrast words ("but", "lekin") to split *progress* from *blocker*

- teammate names mentioned, to find who to relay to
- availability ("free after 6 pm"), turned into a time window

Guardrails (`_finalize`): run on **every** answer, from an LLM or the offline engine. The behaviour is the same whichever layer answered:

Rule	Effect
Partial progress + blocker	"API connected, but waiting for keys" → `Blocked`, never `Complete`
Helping is not a blocker	"I sent the keys to Kabir" does not mark the sender as Blocked
Contradiction guard	Can't complete a task whose upstream is still Blocked (or not Complete in strict mode). Completion is refused with an explanation.
No false refusals	If an LLM refused a clear "done" where no rule applies, the code overrules it
Relay	A blocked task always gets a valid relay target and message when a teammate is involved
Auto-unblock	Only unblocks a real Blocked task owned by someone else
Cleaning	Status names are normalised, confidence is clamped to 0 to 1, teammate names are validated

Other AI features in this file

- `generate_project_report()`: health score, critical-path bottlenecks, cascading risks, handoffs, next actions, per-member standup.
- `generate_project_plan()` / `generate_repo_plan()`: create tasks and owners from a description or from a GitHub repo analysis, with an offline template fallback.
- `coordinate_team_slots()`: finds a shared meeting time from everyone's availability.
- `match_task()`: picks which of a teammate's tasks a message is about, using keyword overlap.

5.5 `telegram_service.py`: the Telegram bot

- Starts in **live** mode when `TELEGRAM_BOT_TOKEN` is valid, otherwise in **Simulator Mode**. The app works the same either way.
- Commands: `/start`, `/auth <PIN>` (proves who you are), `/join <Name>`, `/status` (task board), `/slot` (availability buttons).
- A free-text message is passed to `main.handle_telegram_text()`, which runs `process_reply()`. If the teammate has several tasks, the bot asks "which task?" with buttons.

5.6 `notifier.py`: outbound messages

- `send_telegram()`: sends with 3 retries for flaky Wi-Fi. If the person isn't linked, it is recorded as "simulated".
- `send_slack()`: mirrors relays and escalations to a Slack channel if a webhook is configured.

5.7 `auth.py`: authentication

- Tokens are `base64(payload).HMAC-SHA256 signature`, valid for 14 days. The secret comes from `AUTH_SECRET` or an auto-generated `.auth_secret` file.
- Passwords are salted and hashed. Compares use `hmac.compare_digest`.
- `require_lead` is a FastAPI dependency that protects Team Lead actions.
- Invite codes and links let members join a project.

5.8 `github_repo.py` and `proof.py`: GitHub and proof of work

- `analyze_repo()` reads the repo's languages, manifests (`package.json`, `requirements.txt`, …), and README, then detects the tech stack. The project planner uses this.
- `recent_commits()` and the GitHub webhook map commit authors to teammates. A commit message like "*Connected webhook, blocked on KMS keys from Priya*" updates the board and pings Priya.
- `proof.py` checks that a pasted commit or PR link exists on the project's repo before a task is marked **GitHub Verified**. Non-code tasks use a deliverable link instead.

5.9 `deadlines.py`

Parses human deadlines ("Today 8 PM", "Day 3") into ISO dates, checks for overdue tasks, and spreads task deadlines evenly between a project's start and due dates.

5.10 `seed.py`

Two demo presets:

- **College:** "College Capstone Payment System" with Kabir (Payment API), Meera (Integration, depends on Kabir), and others.
- **Company:** a FinTech release sprint with company roles (Rohan, Priya, Vikram, …).

---

6. Frontend, file by file

6.1 Entry and shell

- **`main.jsx`** wraps the app: `ErrorBoundary` → `ToastProvider` → `AuthProvider` → `App`.
- **`App.jsx`**
  - Shows `LoginScreen` if the user is not signed in, or opened an invite link for another project.
  - `useMobileMode()` shows the phone app for `?mode=mobile`, `/app`, or screens narrower than 768 px. Otherwise it shows the desktop dashboard.
  - Holds the modals (create project, team, tasks, deadlines, integrations, QR connect, Slack).

6.2 Live data: `hooks/useLiveState.js`

1. Fetches a snapshot from `GET /api/state`.
2. Opens a WebSocket to `/ws/live`. On a `STATE_UPDATED` event it re-fetches, debounced by 40 ms.
3. Reconnects with **exponential backoff**, capped at 8 s.
4. Sends a heartbeat ping every 15 s to measure latency.
5. **Polls every 8 s** while the socket is down, so the UI never goes stale.

6.3 `lib/`

File	Purpose
`api.js`	Small `fetch` wrapper. Adds the `Bearer` token and turns FastAPI errors into readable messages.
`auth.jsx`	React context for the session, stored in `localStorage`
`skin.jsx`	Wording per workspace: College ("Teammates", "Submission Deadline") vs Corporate ("Engineering Roster", "SLA")
`tasks.jsx`	`canUpdate()` permission rule (lead: all tasks, member: own tasks) and the verification badge
`conversation.js`	Turns raw `AgentLog` rows into a chat-style feed (incoming, outgoing, action, warning)
`suggestions.js`	One-tap demo replies for the Quick Reply Tester
`utils.js`	Helpers: icons, task numbering, cascading-risk calculation

6.4 `components/`

Component	What it shows
`DashboardView.jsx`	Main desktop board: tasks, statuses, dependency flow, at-risk flags
`IconRail.jsx`	Left navigation rail
`LimePanel.jsx`	Side panel with the coordinator feed and quick actions
`ReportView.jsx`	Standup & Risk Report
`MobileAppView.jsx`	Android companion app: my tasks, report issue, voice update, availability, one-tap unblock banner
`LoginScreen.jsx`, `LoginShowcase.jsx`	Sign in, register lead, join with invite, demo login
`PhoneConnect.jsx`, `modals/QRConnectModal.jsx`	QR codes for phones and Telegram
`modals/*`	Create project (manual, AI, or from GitHub), team, task update, deadlines, feed, integrations, Slack
`Toast.jsx`, `ErrorBoundary.jsx`, `Modal.jsx`, `ui.jsx`	Shared UI building blocks

6.5 `vite.config.js`

Serves on `0.0.0.0:5173` so phones on the same Wi-Fi can connect. It proxies `/api` and `/ws` to the backend, so the phone only needs one address.

---

7. One message, end to end

Kabir types on his phone: *"The API is connected, but I'm waiting for test credentials from Meera."*

1. **Mobile app** → `POST /api/mobile/quick-update`.
2. `process_reply()` loads the context and calls the **AI engine**.
3. **Layer 1 (Groq)** returns JSON. If it's down, layers 2, 3, or 4 answer instead.
4. `_finalize()` **guardrails:** the message has progress *and* a blocker, so the status is `Blocked`, the progress note is "API is connected", the blocker is "Waiting for test credentials", and the relay target is Meera.
5. **DB update:** Kabir's task becomes `Blocked`. Meera's dependent task shows as **At Risk**.
6. **Relay:** Meera gets a Telegram message: "Kabir is blocked… could you send the test credentials?"
7. **Middleware** broadcasts `STATE_UPDATED`, and every screen refreshes.
8. Later Meera says *"Integration is done!"*. The **contradiction guard** refuses it, because the upstream task is still Blocked.
9. Meera says *"Sent the test credentials to Kabir"*. **Auto-unblock** moves Kabir's task to `In Progress` and notifies him.

---

8. Running and testing

```
# Setup
cd backend && python -m venv venv && venv\Scripts\activate && pip install -r
requirements.txt
copy .env.example .env
cd ../frontend && npm install

# Run both (from project root)
python start_all.py            # --open to launch the browser, --check for a smoke
test
```

URL	What
`http://localhost:5173`	Desktop dashboard
`http://<LAN_IP>:5173/?mode=mobile`	Phone app
`http://localhost:8000/docs`	Swagger API docs

Tests

- `python verify_demo.py` runs the full demo story over HTTP against the live API and prints a pass/fail table. It starts a temporary backend if none is running. Use `--offline` to test without LLM keys.
- `python backend/self_test.py` is the backend test suite. It uses FastAPI's `TestClient` on a throwaway SQLite DB, always tests the offline engine, and tests the live LLMs too when keys are set.

---

9. Design decisions worth mentioning

Decision	Why
Guardrails in code, not in the prompt	LLMs can be inconsistent. Enforcing the rules in Python gives the same behaviour from every AI layer.
4-layer AI fallback	The live demo can't depend on Wi-Fi or free-tier rate limits
One `process_reply()` for every channel	Telegram, mobile, dashboard, and GitHub all get exactly the same logic
Broadcast from middleware	No route can forget to push a live update
SQLite + in-place migrations	Zero setup, a single file, and old data survives schema changes
Vite proxy + LAN binding	Phones need only one URL. No CORS or port juggling.
Simulator Mode	Works with no Telegram token. Messages are logged as "not sent" instead of failing.
Polling fallback for the WebSocket	The UI stays correct even if the socket drops